CEJOR (2015) 23:815–832
DOI 10.1007/s10100-014-0362-7

ORIGINAL PAPER

Optimal circle covering problems and their applications

Balázs Bánhelyi · Endre Palatinus · Balázs L. Lévai

Published online: 18 September 2014
© Springer-Verlag Berlin Heidelberg 2014

Abstract We study a special type of circle covering problem, the complete cover of polygons by not necessarily congruent circles with prescribed centres. We introduce a branch-and-bound algorithm which is able to check whether an actual set of circles covers a given polygon. Furthermore, we present a method capable of finding the optimal cover with arbitrary precision. These techniques are based on interval arithmetic, therefore our results are numerically reliable. To demonstrate the performance of the developed optimization method, we determine the optimal cover of the unit square and an irregular polygon.

Keywords Interval arithmetics · Circle covering · Global optimization

1 Introduction

The circle packing problem has attracted much attention in the last century, and special variants (Kubach et al. 2009) like packing of equal circles in a square(Huang and Ye

E. Palatinus: work done while at the University of Szeged.

The publication is supported by the European Union and co-founded by the European Social Fund. Project title: "Telemedicine-focused research activities on the field of Mathematics, Informatics and Medical sciences". Project No.: TÁMOP-4.2.2.A-11/1/KONV-2012-0073.

B. Bánhelyi (✉) · B. L. Lévai
Institute of Informatics, University of Szeged, Szeged, Hungary
e-mail: banhelyi@inf.u-szeged.hu

B. L. Lévai
e-mail: levaib@inf.u-szeged.hu

E. Palatinus
Information Systems Group, Saarland University, Saarbrücken, Germany
e-mail: palatinus@cs.uni-saarland.de

2011) receive much attention even nowadays. The objective of this task is to find the densest packing of a given number of congruent circles with disjoint interiors in a unit square.

Circle covering is the dual problem of circle packing. It has turned out to be much more difficult than the packing problem. The fewer number of publications on this topic indicates this well. Usually, research papers improve the best known bound for a packing or covering without any proof of optimality, or on the other hand, the authors verify the optimality of a particular circle configuration. Considerable many papers are available about circle covering on the web (Friedman 2014). For more numerical results, see Nurmela (2000); however, we remark that it lacks the proof of optimality, and the results cannot be considered valid due to the uncertain numerical computations.

In this paper, we aim to find the thinnest covering of polygons with a given number of circles while overlapping interiors are allowed. By thinnest we mean the total covering of a polygon with circles of minimal radii.

We developed two branch-and-bound based methods. The first one serves cover checking purposes, the second one is able to optimize the cover. In both techniques, interval arithmetic is used to overcome such computational limitations as the imprecision of fixed-point arithmetic, hence our results are numerically reliable.

The efficiency of our methods are demonstrated on the covering of the unit square and on a telecommunication related problem. Knowing the locations of the main broadcasting antennas, we would like to cover the country of Hungary with TV stations for terrestrial signal. A region covered by a single station can be considered more or less of circular shape. Since the area is proportional to the energy emission, the goal is to minimize the area of the union of these regions. Similar problems were solved in the paper (Das et al. 2006), although it also applied unreliable computations.

Some of the theoretical and algorithmical techniques in the following sections have already been successfully applied in the field of circle covering (Palatinus 2010).

2 Theoretical background

We consider a specific type of circle covering problem assuming that the centres of the circles are already fixed and known, only the radii are allowed to be changed. In this section, we introduce the key definitions and notations.

Definition 2.1 $O_i = (x_i, y_i, r_i)$ denotes the open circle i, where x_i, y_i are the coordinates of its centre and r_i represents its radius, while $\mathcal{O} = \{O_i : i = 1, \ldots, n\}$ is the set of arbitrarily given circles.

Definition 2.2 The configuration r belonging to the set of circles $\mathcal{O}$ is the vector $(r_1, r_2, \ldots, r_n)$, where r_i is the radius of $O_i \in \mathcal{O}$.

Our method is applicable when the region we are covering, henceforth referred to as R, is closed, bounded, and connected. R is a square in most cases, but the cover of rectangles (Heppes and Melissen 1997), circles (Heppes 2006), regular triangles (Nurmela 2000), and other shapes have been studied as well. Now, we define when we consider a two-dimensional shape covered by a set of circles.

Definition 2.3 We say that the configuration r of the circle set $\mathcal{O}$ is good (it is a covering one) regarding the region R if and only if the following condition is satisfied:

$$\forall (x, y) \in R \; \exists \, O_i \in \mathcal{O} : \; d((x, y), (x_i, y_i)) = \sqrt{(x - x_i)^2 + (y - y_i)^2} < r_i,$$

where function $d(.,.)$ is the Euclidean distance. We say that the configuration r is bad otherwise.

It is important to emphasize that the distance of a point and a centre of circle has to be strictly lower and not lower or equal than the radius because the circles are open.

We aim to provide solutions with mathematical precision. The application of interval arithmetic ensures to get verified results and to have a computer aided proof of validity.

Definition 2.4 A one dimensional, compact interval, or simply an interval, is spanned by two real numbers:

$$X = [\underline{X}, \overline{X}] = \{x \in \mathbb{R} : \underline{X} \leq x \leq \overline{X}\},$$

where $\underline{X}$ is the lower bound and $\overline{X}$ is the upper bound of the interval.

Definition 2.5 An n dimensional interval is a vector $X = (X_1, X_2, \ldots, X_n)$, where every X_i $(i = 1, \ldots, n)$ is a one dimensional interval. Multidimensional intervals are also called boxes.

Let $\mathbb{I}$ be the set of compact intervals in $\mathbb{R}$ and $\mathbb{I}^n$ be the set of n dimensional intervals.

Definition 2.6 The width of a one dimensional interval is:

$$\text{Width}(X) = \overline{X} - \underline{X}.$$

The concept can be naturally generalized to n dimension:

$$\text{Maximal width(X)} = \max_{\text{i=1,...,n}} (\overline{\text{X}}_\text{i} - \underline{\text{X}}_\text{i}).$$

Because we will not use any other kind of width in the paper, we will simply refer to the extended concept as width instead of maximal width.

Interval computations use intervals instead of real numbers, therefore we have to extend the well-known arithmetical operators to intervals.

Definition 2.7 Let Ω be the set of the elementary real operators. These operators can be extended to be interval operators by the following definition:

$$A \circ B = \{a \circ b : a \in A \text{ and } b \in B\},$$

where $A, B \in \mathbb{I}, \circ \in \Omega$.

By the above definition, computing the result of any interval operation would require the execution of an infinite number of real operations when both operands have a non zero width. Fortunately, a finite number of operations are sufficient in certain cases if we take advantage of the monotony of the operators. More information is available about interval arithmetic in Alefeld and Herzberger (1983); Alefeld and Mayer (2000). During our research, we used the *C-XSC* library (http://www.xsc.de/) for implementation.

3 The verifier algorithm

Our task is to decide whether a particular configuration is good in the sense of Definition 2.3. In order to verify that a configuration covers a region, we validate the following goodness property point by point.

Definition 3.1 The point $p = (x, y)$ of the region R has the property $\mathcal{P}$ regarding the configuration r if and only if at least one of the circles of r covers p. Formally:

$$\exists\, O_i \in \mathcal{O} :\ d((x, y), (x_i, y_i)) = \sqrt{(x - x_i)^2 + (y - y_i)^2} < r_i,$$

where function $d(.,.)$ is the Euclidean distance, and $\mathcal{O}$ is the set of circles r belongs to.

We can simply extend the property $\mathcal{P}$ for two-dimensional intervals. If every point of an interval has this property, then the interval itself has it too. Algorithm 1, a branch-and-bound algorithm, performs the checking of configurations. The algorithm starts from a bounding interval of R allowed to be larger than R and being not necessarily its convex hull. It partitions the intervals until it does not found a counterexample to the cover, or every generated subinterval having common points with R holds the property $\mathcal{P}$. An interval is considered to be a counterexample if it has a smaller width than a predefined ε value.

The following theorem guarantees the correctness of Algorithm 1.

Theorem 3.2 *Assume that we use an interval extension of a verifier algorithm to the property $\mathcal{P}$, and this algorithm returns that every point of R has the property $\mathcal{P}$. In this case, Algorithm 1 creates a partition of the initial bounding interval of R in which for every subinterval at least one of the following conditions holds:*

1. *the subinterval does not overlap with the studied region R, or*
2. *at least one circle contains all the points of the subinterval.*

The next objective is to validate that the algorithm is able to decide whether the studied region is totally covered.

Theorem 3.3 *Assume, that we have an interval extension of a verifier algorithm to the property $\mathcal{P}$, and it is capable of determining whether a point p and some neighbourhood of it have the property $\mathcal{P}$; furthermore, the parameter ε of Algorithm 1 is set to zero, and every point of R has the property $\mathcal{P}$. In this case, Algorithm 1 will terminate with a positive result after a finite number of iterations.*

Algorithm 1 Verifier algorithm

Input

ε: the minimal allowed size of subintervals.
R: the studied region.
$\mathcal{O}$: the set of covering circles.

Output

An interval violating the property $\mathcal{P}$ is shown with a size less than ε.

Auxiliary data structures

S: a stack.

Return value

Boolean value: it indicates whether every point of R has the property $\mathcal{P}$.

```
1: I ← a bounding interval of R
2: PUSH I into S
3: while S is not empty do
4:     POP V from S
5:     if R ∩ V ≠ ∅ and P do not holds for V regarding O then
6:         if width(V) < ε then
7:             print V
8:             return false
9:         else
10:            H1, H2 ← two halves of V cut along the largest dimension
11:            PUSH H1 into S
12:            PUSH H2 into S
13:        end if
14:    end if
15: end while
16: return true
```

We still need to study how the algorithm operates when the examined region does not have the property $\mathcal{P}$.

Theorem 3.4 *Assume that we have an interval extension of a verifier algorithm to the property $\mathcal{P}$, the parameter ε of Algorithm 1 is set to zero, and there is a point $p \in R$ which violates the property $\mathcal{P}$. In this case, Algorithm 1 cannot terminate after a finite number of steps.*

Theorems 3.2–3.4 can be proved analogously as Theorems 2–3 in Csendes et al. (2006).

The discussed algorithm strongly relies on the determination of two important features of two-dimensional intervals. First, we need to know whether an interval overlaps the studied region. Second, we also have to check if every point of an interval has the property $\mathcal{P}$.

Algorithm 2 Interval in Polygon algorithm

Input

P: the array containing the sides of an n-sided polygon. The sides, determined by endpoints, follow each other in the same order as they do in the polygon. P_i denotes the ith element of P.
$V = ([x_1, x_2], [y_1, y_2])$: the studied two-dimensional interval.

Return value

Boolean value: it indicates whether P and V have a common point.

1: **if** $\exists i \in 1, \ldots, n : P_i \cap V \neq \emptyset$ **then**
2: **return true**
3: **end if**
4: $m \leftarrow 0, i \leftarrow 1, state \leftarrow out, visited \leftarrow 1$
5: $V_b' \leftarrow ([-\infty, x_1], [-\infty, y_2]), V_t' \leftarrow ([-\infty, x_1], [y_1, \infty])$
6: **if** $\forall i \in 1, \ldots, n : P_i \cap (V_t' \cup V_b') = \emptyset$ **then**
7: **return false**
8: **end if**
9: $i \leftarrow$ *the first index j such that* $P_j \cap (V_t' \cup V_b') \neq \emptyset$
10: **while** $visited \leq n$ **do**
11: **if** $P_i \cap V_b' \neq \emptyset$ **and** $P_i \cap V_t' \neq \emptyset$ **then**
12: $m \leftarrow m + 1$
13: **else if** $state = out$ **and** $P_i \cap V_b' \neq \emptyset$ **then**
14: $state \leftarrow top$
15: **else if** $state = out$ **and** $P_i \cap V_t' \neq \emptyset$ **then**
16: $state \leftarrow bottom$
17: **else if** $state = top$ **and** $P_i \cap V_t' \neq \emptyset$ **then**
18: $m \leftarrow m + 1, state \leftarrow out$
19: **else if** $state = bottom$ **and** $P_i \cap V_b' \neq \emptyset$ **then**
20: $m \leftarrow m + 1, state \leftarrow out$
21: **end if**
22: $i \leftarrow i + 1 \mod n, visited \leftarrow visited + 1$
23: **end while**
24: **if** m is $even$ **then**
25: **return true**
26: **else**
27: **return false**
28: **end if**

Algorithm 2 checks the existence of overlapping. One can consider it the interval extension of Shimrat's *Point in Polygon* algorithm Shimrat (1962), since it is based on the same idea. Let the region R be a polygon and $V = ([x_1, x_2], [y_1, y_2])$ be the two-dimensional interval we study. Practically, we have to count how many side sequences of R are left of V. If this number is odd, then R and V overlap. The two objects are disjoint otherwise.

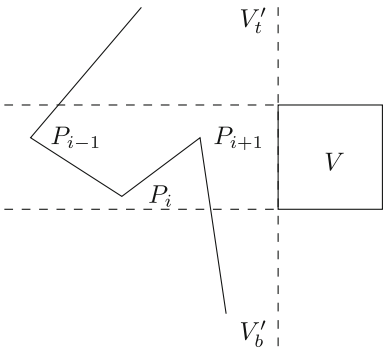


Fig. 1 Illustration of the special regions used in Algorithm 2

The verifier starts by testing two trivial conditions. If there is a side of the polygon already having a common point with V, then the objects clearly overlap. If there is no such side, and the polygon is located entirely left and above of V (in the region V'_t), or left and below of V (in the region V'_b), then the interval and the polygon are obviously disjoint. For illustration, see Fig. 1. After the initial tests, we can start counting. The most difficult part is to determine where the sequences are started, where they are ended, and which direction the currently followed sequence is heading, but these are rather technical problems than real algorithmic challenges.

The interval extension makes us face some interesting situations which cannot occur with the classic Point in Polygon algorithm. For example, a two dimensional interval may have inner and boundary points with respect to a specific side, when one side of the polygon crosses the interval, or it could even contain a whole sequence of polygon sides.

The verification of the property $\mathcal{P}$ is much simpler. For every point of V, we only need to test whether there is at least one circle whose centre is closer to the point than its radius. Formally,

$$\exists \mathcal{O}_i \in \mathcal{O}, \forall (x, y) \in V, \sqrt{(x - x_i)^2 + (y - y_i)^2} < r_i$$

must be true. The algorithm called *Point in Circle* executes this task. We implemented it according to the strategy we have just introduced, although its operation requires some additional discussion. First, one may recognize that Algorithm 3 is unable to handle situations when more than one circle realize the total cover of an interval. In other words, the interval is entirely covered but none of the circles contain it. Second, if the interval is uncovered, it remains unknown that exactly which part of the interval violates the cover.

However, it might seem troubling that Algorithms 2 and 3 only indicates which interval needs further examination, Algorithm 1 will continue precisely the partitioning of the cases where the property $\mathcal{P}$ may hold. In this way, Algorithms 2 and 3 perfectly fit their roles.

Theorem 3.5 *If a polygon P is covered by circles, then Algorithms 1, 2, and 3 terminate after a finite number of iterations with a positive outcome when ε is set to zero.*

Proof It directly follows from Theorem 3.3. □

Algorithm 3 Point in Circle algorithm

Input

$\mathcal{O} = \{(x_i, y_i, r_i) : i = 1, \ldots, n\}$: the set of circles.
$V = ([x_1, x_2], [y_1, y_2])$: the studied two-dimensional interval.

Return value

Boolean value: it indicates whether V is contained by any of the input circles.

```
1: for all O_i in O do
2:     if max_{p∈V}(dist(p, (x_i, y_i))) < r_i then
3:         return true
4:     end if
5: end for
6: return false
```

As a remembrance, we would like to emphasize that the verifier only fits the positive cases. In other words, it does not terminate when the polygon is not totally covered (see Theorem 3.4). We also have to mention that we might have a negative result even in case of a positive ε value, but this does not mean undoubtedly that the studied geometrical condition is not satisfied.

We worked with open circles which means that their borders do not take part in the covering. This is what makes it possible to verify the cover of any point and its appropriately small neighbourhood. We would be unable to achieve this if we used closed circles. For example, consider the contact of two covering circles as a point of the region R. The point itself is covered, but this is not true for any of its neighbourhood. Our algorithms could not handle such cases, though it would be necessary for Theorem 3.3.

This cover checking problem was approached as general as possible. There are more efficient algorithms to check whether a polygon is covered if some special conditions are met, like the centres of the circles are located at the vertices of the polygon Casado et al. (2011), but we did not intend to optimize for such special cases. Also, Algorithm 3 could be extended to determine the intersections of the intervals and the circles in order to decide which parts are covered and which are not, and we could count how many circles covers the points instead of simply checking the cover, but we did not need this extra information in the later phases of our study.

4 Results of the verifier algorithm

We take a look at the possible outcomes of the verifier algorithm. Figure 2 shows two different runs. In both presented cases, we studied the cover of the polygon approximation of the country of Hungary. The initial bounding intervals were the large rectangles containing the whole polygon. The verifier algorithm partitioned these rectangles until the created subintervals fulfilled the stopping criterion.

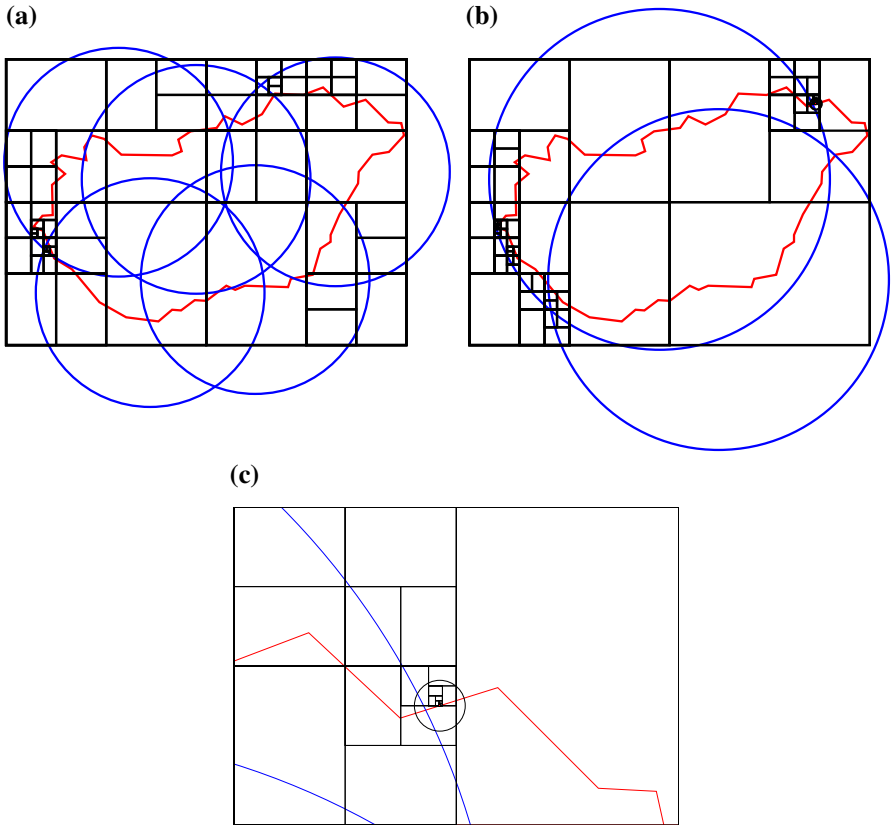


Fig. 2 Subintervals generated by the verifier algorithm on two specific circle configurations. **a** Image of a positive case. **b** Image of a negative case. **c** Zoomed in the upper-right corner of **b**

On Fig. 2a, the algorithm found the cover complete. The generated subintervals do not overlap the polygon (see rectangles in the corners), or they do, but one or more circles totally cover them (see rectangles in the middle).

Figure 2b shows a case when the algorithm returned a counterexample to the cover located in the small black circle in top right corner, also illustrated in more detail on Fig. 2c. The width of this subinterval is smaller than the given ε value, and no circle covers it completely. One may notice that a lot of iterations had to be made before this interval was found.

An interesting effect of interval arithmetic is that some intervals were subdivided despite they were clearly out of the polygon. Although they are illustrated in this way, their bounding intervals were larger due to the over-estimation of interval expressions, and this indicated a common part with the polygon during the run of the algorithm.

The total runtimes on these examples were just a few seconds. We have also experimented with the parallelization of the algorithm, and we have been successful in reaching a linear speed-up with respect to the number of CPU cores. This improvement serves the goal to make it possible to place the verifier algorithm into a more

complex framework, like a global optimizer method which is able to find the optimal cover of a polygon.

5 The cover optimizer algorithm

In many practical applications, devices require power proportional to the area they cover. More precisely, the power requirement is proportional to the square of the operational radius. The examined emitters and sensors worked exactly in this way. Our goal is to reduce the energy usage of a set of such devices as much as possible by minimizing the sum of squares of the operational radii. Formally, we have to find the minimum of the objective function:

$$f(r_1, r_2, \ldots, r_n) = \sum_{i=1}^{n} r_i^2, \tag{1}$$

where $r_1, r_2, \ldots, r_n$ form a good configuration in the sense of Definition 2.3. Goodness and badness will hence be interpreted during the whole discussion of the optimizer algorithm.

First of all, we prove some simple configuration related lemmas we will rely on later.

Lemma 5.1 *If a configuration $r = (r_1, r_2, \ldots, r_n)$ is good, then every configuration $r' = (r'_1, r'_2, \ldots, r'_n)$ will also be good, where $r'_i \geq r_i$ for $i = 1, 2, \ldots, n$.*

Proof The initial r configuration is a good one, i.e. it covers the aimed region. Obviously, replacing some circles by larger ones in the configuration does not destroy the covering property. □

It is not too difficult to phrase similar statements for the alteration of bad configurations.

Lemma 5.2 *If a configuration $r = (r_1, r_2, \ldots, r_n)$ is bad, then every configuration $r' = (r'_1, r'_2, \ldots, r'_n)$ configuration will also be bad, where $r'_i \leq r_i$ for $i = 1, 2, \ldots, n$.*

Proof Analogous to the proof of Lemma 5.1. □

One may realize, based on these lemmas, that certain configurations can represent an entire set of configurations. Therefore it is advantageous to handle these sets together.

Definition 5.3 Let $C = \left[[\underline{c_1}, \overline{c_1}], \ldots, [\underline{c_n}, \overline{c_n}]\right]$ be a set of configurations whose ith component is in the interval $[\underline{c_i}, \overline{c_i}]$ for every $i = 1, 2, \ldots, n$. We call $\underline{C} = (\underline{c_1}, \underline{c_2}, \ldots, \underline{c_n})$ and $\overline{C} = (\overline{c_1}, \overline{c_2}, \ldots, \overline{c_n})$ the extremal configurations of C.

We illustrated the extremal configurations, Lemmas 5.1, and 5.2 on Fig. 3a-c respectively.

Using the new notations, let us examine the relation between optimal and extremal configurations.

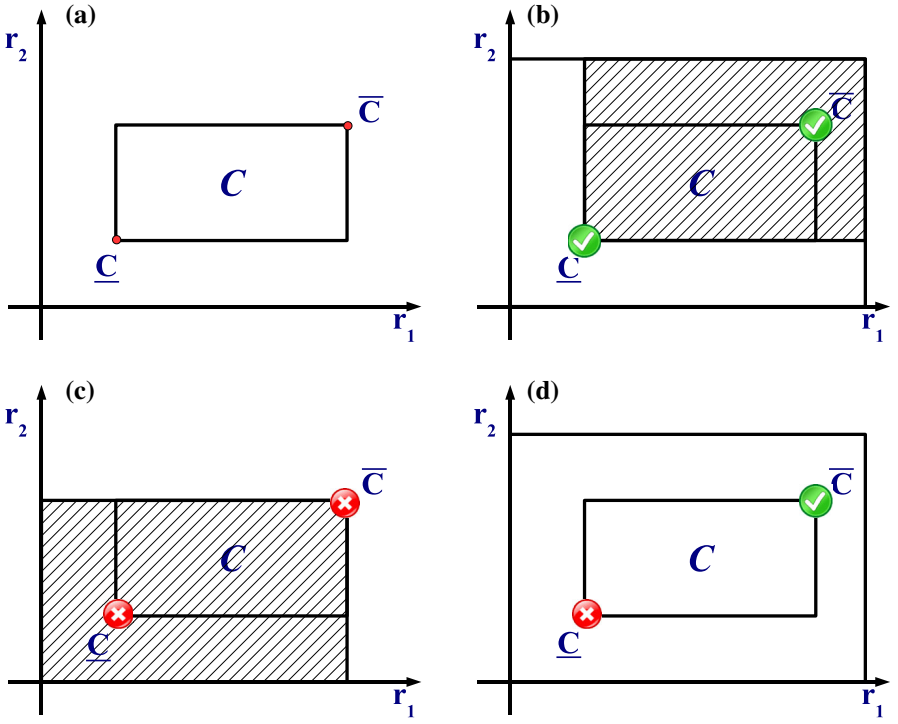


Fig. 3 Illustration of the connection between the extremal configurations and the goodness of configuration sets. Configurations consist of two circles with radii r_1 and r_2. **a** Extremal configurations of a set. **b** A set of good configurations. **c** A set of bad configurations. **d** A configuration set which may contain an optimum

Theorem 5.4 *Any set of configurations C can only contain an optimal configuration in its interior if and only if $\underline{C}$ is a bad configuration and $\overline{C}$ is a good one.*

Proof Assume, that the extremal configuration $\overline{C}$ of the configuration set C is bad, but C contains optima. We can create all the elements of C by shrinking appropriately the radii in $\overline{C}$. According to Lemma 5.2, such alterations result only in bad configurations. This implies that every element of C is bad, which is a contradiction since we asserted that C contains optimal configurations, which are good, therefore $\overline{C}$ must be good.

We continue by assuming that the extremal configuration $\underline{C}$ is good, and C still contains optima. It follows from the structure of the objective function f, defined in Eq. (1), and from definition 5.3 that $\underline{C}$ has the lowest objection function value in C yielding that there is one and only one optimum in C, and this configuration is $\underline{C}$. Then let $\underline{c_i} = (\underline{x_i}, \underline{y_i}, \underline{r_i})$ be the ith circle of $\underline{C}$ such that some points of the studied region R are covered only by this circle. If there were no such circle, this would mean that every point is covered at least doubly, therefore we could reduce the radius of a circle to zero still covering every point. This would contradict that $\underline{C}$ is optimal as the newly created configuration would be obviously better. Therefore we are able to select such a circle. Let us denote by A the region that is only covered by $\underline{c_i}$. The circles are open while R is closed, thus A is closed, and we can also determine a sufficiently small

positive ε for which the circle $c_\varepsilon = (\underline{x_i}, \underline{y_i}, \underline{r_i} - \varepsilon)$ still covers A. If we replace $\underline{c_i}$ with c_ε in $\underline{C}$, the created configuration

$$C_\varepsilon = (\underline{c_1}, \ldots, \underline{c_{i-1}}, c_\varepsilon, \underline{c_{i+1}}, \ldots, \underline{c_n})$$

remains good. The radii are equal in C_ε and $\underline{C}$ except that $r_\varepsilon \leq c_i$. This implies that $f(C_\varepsilon) < f(\underline{C})$ holds, thus $\underline{C}$ cannot be optimal leading us to a contradiction again.

We have shown that $\overline{C}$ must be good and $\underline{C}$ must be bad in order to C can contain optimal configurations, as illustrated on Fig. 3d, the proof is complete. □

It follows directly from the proof that optimal configurations only exist as limits. The result above can be generalized if the objective function is altered in a way that preserves its monotony, and the new concepts of goodness and badness remain monotone in a suitable way. For example, it would be a simple extension to consider the 3-dimensional space using spheres for covering and a cubic expression instead of a quadratic one as an objective function.

Lemma 5.5 completes the theoretical background to construct an optimizer algorithm.

Lemma 5.5 *There is no such configuration set C for which $\underline{C}$ is a good configuration and $\overline{C}$ is a bad one.*

Proof This lemma is a trivial consequence of Lemma 5.1. □

Let us discuss Algorithm 4, the cover optimizer. Using the branch-and-bound strategy to search for an optimal cover seemed promising again as this approach was studied in Cambini and Sodini (2008) and applied successfully in similar situations Bánhelyi et al. (2008). We add only the sets which potentially contain optimal configurations based on Theorem 5.4. We always take out the set with the best lower bound of the objective function to ensure the global optimality of the found configuration (see Theorem 5.6). Instead of a priority queue, we could use other selection methods like pf^* in Casado et al. (2000) and Csendes (2001), but checking the stopping condition would be unnecessarily more complicated.

As we do not aim to find all the optima, the search stops as soon as an approximation emerges at the user-defined level of ε. It is possible, or even certain when some symmetry is present in the problem, that other optima are in the priority queue when we finish the search, but none of them is better than the found one by more than $\varepsilon\%$.

Additionally, we could use an absolute limit at the start of the loop phase to terminate. In this case, the returned solution would differ from the real global optimum by at most ε. We decided to use the relative term because the relative difference and the global optimum are invariant to scaling while the absolute difference is not. Some cases among our examples will show a great variety in magnitude proving the soundness of this choice.

We based Algorithm 4 on the idea of traditional interval optimizer frameworks (Pál and Csendes 2009; Ratschek and Rokne 1988). Unfortunately, neither accelerating techniques, like Newton's method (Ratschek and Rokne 1988; Vinkó and Ratz 2004), nor complexity reducing transformations, such as the ones in Antal et al. (2013), can be applied in our case due to our optimization constrains.

Algorithm 4 Optimizer algorithm

Input

ε: precision parameter, the maximal relative difference between a result and the real optimum in percentage.
C: the configuration set, the possible radius magnitudes.

Auxiliary data structures

Q: a priority queue popping the element at which the objective function f, defined in Eq. (1), is minimal.

Return value

configuration: it is worse than the real optimum by at most $\varepsilon\%$.

if $\overline{C}$ is *good* **then**
  PUSH C onto Q
end if
loop
  POP C from Q
  $F \leftarrow$ bounding interval of objective function values on C
  if $width(F)/\overline{F} \cdot 100 < \varepsilon$ **then**
    return $\overline{C}$
  else
    Cut C into halves along the largest dimension
    $C_1 \leftarrow$ the half of C which contains $\underline{C}$
    $C_2 \leftarrow$ the half of C which contains $\overline{\overline{C}}$
    if $\overline{C_1}$ is *good* **then**
      PUSH C_1 INTO Q
    end if
    if $\underline{C_2}$ is *bad* **then**
      PUSH C_2 INTO Q
    end if
  end if
end loop

The following theorem states the global optimality of the returned configurations.

Theorem 5.6 *Assume that we have a verifier algorithm capable of deciding the goodness of configurations. In this case, the optimizer algorithm always returns a good configuration, and there is no other good configuration with an objective function value better by more than $\varepsilon\%$.*

Proof Assume that the algorithm stopped and returned a configuration, but the real global optimum is better than it by more than $\varepsilon\,\%$. Let r_o and r_g denote the found and the globally optimal configurations while C_o and C_g denote the sets which contain them. The algorithm discarded precisely those configuration sets which could not contain

global optima according to Theorem 5.4. We pushed every other configuration set into the priority queue including C_g.

Remember that $f(r)$ denotes the objective function value at the configuration r. Let us extend this notion to configuration sets and denote the one-dimensional bounding interval of the set $\{f(r) : r \in C\}$ by $F(C)$, where C is an arbitrary configuration set. The algorithm stopped because C_o satisfied the following condition:

$$\frac{\text{Width}(F(C_o))}{\overline{F(C_o)}} \cdot 100 < \varepsilon. \tag{2}$$

Applying Definition 2.6 in (2) and rearranging the two sides, we obtain

$$\frac{\overline{F(C_o)}}{\underline{F(C_o)}} < \frac{100}{100 - \varepsilon}. \tag{3}$$

Since r_o is the upper extremal configuration of C_o, and the priority queue pops the configuration C which has the lowest $\underline{F(C)}$ value, the following inequalities hold:

$$f(r_o) \leq \overline{F(C_o)}, \tag{4}$$

$$f(r_g) \geq \underline{F(C_g)} \geq \underline{F(C_o)}. \tag{5}$$

Substituting $f(r_g)$ in the denominator and $f(r_o)$ in the nominator in the left hand side in (3),

$$\frac{f(c_o)}{f(c_g)} < \frac{100}{100 - \varepsilon} \tag{6}$$

holds according to (4) and (5). This contradicts what we initially assumed. The proof remains valid if more than one globally optimal configurations exist. We leave it to the reader to verify that. □

6 Results

Our first experiment aims to cover the unit square by two circles. The operation of the optimizer is illustrated on Fig. 4a. The points represent configurations of two circles with radii r_1 and r_2. Both are at most 2 as this is denoted on the axes. The shaded rectangles are the generated configuration sets. The dark grey and light grey sets were discarded during the search because the former had bad upper extremal configurations while the latter had good lower extremal configurations in the sense of Definition 2.3. The found optimum is located in the small circle, also shown in more detail on Fig. 4b. The grey curves represent different levels of objective function values. The thick one denotes configurations having the same value as the optimum has. If the returned configuration was not globally optimal, then we should continue to search in the white squares which have better objective function values.

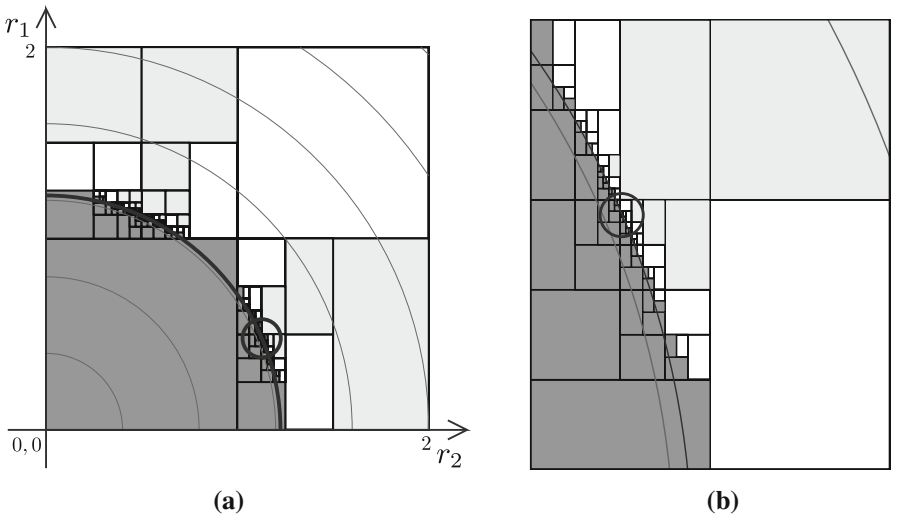


Fig. 4 Generated subintervals illustrating how the optimizer algorithm operates. **a** All generated subintervals. **b** Zoomed in the optimum of **a**

Figure 5a shows the positions of the centres, the radii, and the computed cover whose numerical analysis is a good opportunity to show what we have already proved theoretically. The precision was set to 1 % during all searches. The optimal radii we found were 1.12890625 and 0.48046875. The value of the objective function f is $\approx$1.50527954. If closed circles were considered, the theoretical optimum would be the minimum of the expression

$$1 + (1 - x)^2 + x^2,$$

where x denotes the radius of the smaller circle. If one rearranges the coefficients, the obtained form is:

$$2(x - 0.5)^2 + 1.5,$$

it is easily seen that the minimum is at $x = 0.5$. Comparing this value to 1.50527954, it is clear that our algorithm worked within the 1 % error bound.

Shown on Fig. 5b, the optimum remained approximately the same when we used three circles instead of two. The radius of the third circle is 0.00390625, which points out an interesting characteristic of the optimizer. By definition, upper extremal configurations consist of upper bounds of radii. Initially, these bounds are positive, and partitioning the intervals preserve this condition forcing every circle in the optimum to have a positive radius even if it is not necessary to obtain a complete cover. It is left to the reader to check that the configuration of this case is also within the prescribed 1 % precision.

Using a fourth circle did not improve the cover. Presented on Fig. 5c, the smaller circles have an equal radius of 0.359375000. The radii of the larger circles are different,

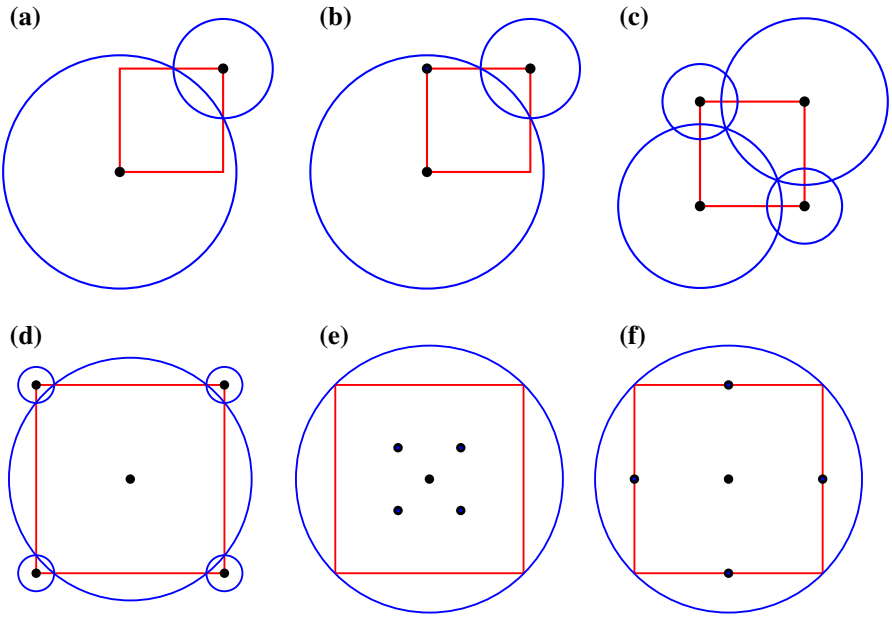


Fig. 5 Interesting optimal covers of the unit square. *Black dots* denote the centres of the circles. **a** Covering from opposite corners. **b** Covering from three corners. **c** Covering from all corners. **d** Covering from the corners and the centre of the square. **e** Covering from five inner points including the centre of the square. **f** Covering from all side midpoints and the centre of the square

0.796875000 and 0.783203125. The objective function value is 1.50671, worse than in the previous cases. As before, the theoretical optimum can be calculated minimizing the expression

$$2(\sqrt{2}/2 - x)^2 + (\sqrt{2}/2)^2,$$

where x is the radius of the smaller circles. Rearranging the coefficients leads us to the optimal value of 1.5 again. Although better configurations were in the priority queue, like the previous ones, the given ε level of precision allowed the optimizer to accept a totally different cover. Had provided a lower ε value, this exotic configuration would certainly have been discarded.

On Fig. 5d, we added the centre of the square to the set of possible centres. Unlike what we conjectured, that the optimum should be a single large circle, the optimal cover consists of five circles with radii 0.095703125 and 0.64453125. The objective function value of this configuration is 0.4520569, much lower than 0.5, the value of our conjecture. Two more interesting sets of centres and the found covers are illustrated on Fig. 5e, f without further discussion.

Finally, here are some examples to show how the optimizer performed when we tried to cover a more complex polygon, the representation of the country of Hungary. The centres were generated randomly with a uniform distribution from the positions of larger Hungarian cities. Figure 6a–c illustrates the optimal covers by four, five, and six circles, respectively. Figure 6d shows an unexpected case when the optimizer

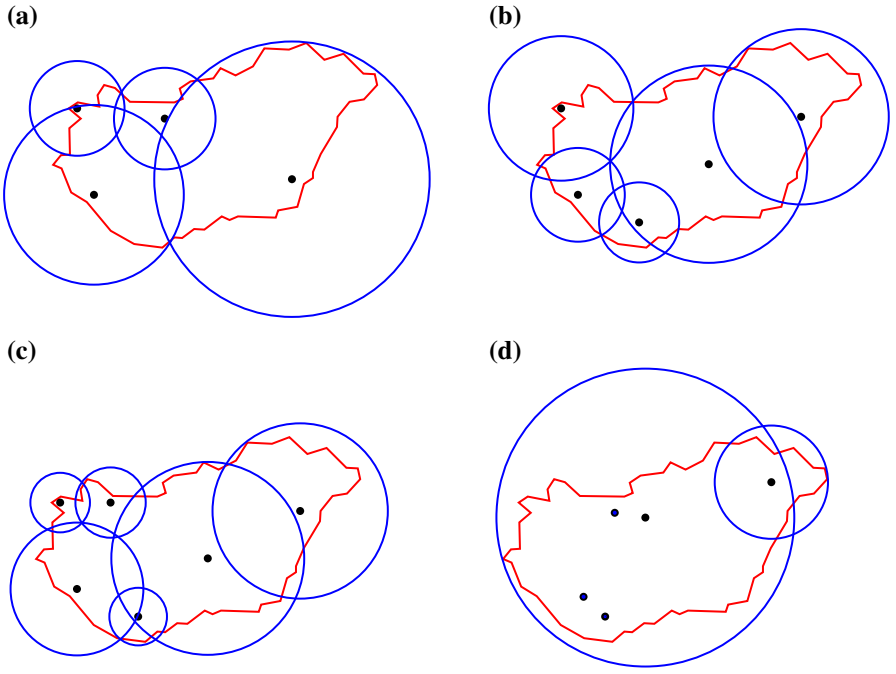


Fig. 6 Some optimal covers of Hungary. *Black dots* denote the centres of the circles. **a** Covering by four circles. **b** Covering by five circles. **c** Covering by six circles. **d** Covering by five circles, using only two effectively

was given 5 centres, but the returned cover used only two of them. Although these solutions are numerically optimal, realizing them is probably impossible due to the range constraints of real TV and radio towers; however, we can get feasible covers if we slightly modify the optimization by maximizing the allowed radii.

7 Conclusion

This paper studied a special type of circle covering problem by reliable numerical tools. The centres of circles were considered to be fixed allowing only the radii to be changed, and we were interested in the thinnest cover of polygons. Two important algorithms were presented, a verifier and an optimizer. The former is capable of validating whether a given set of circles covers a polygon, while the latter aims to find the cover closest to the global optimum with arbitrary precision. Both algorithms are based on interval arithmetic, thus the provided results are mathematically reliable and free of numerical errors. To demonstrate their operation and effectiveness, we calculated several optimal covers of the unit square and the polygon approximation of the country of Hungary using various number of circles and different centres.

References

Alefeld G, Herzberger J (1983) Introduction to interval computations. Academic Press Inc., New York
Alefeld G, Mayer G (2000) Interval analysis: theory and applications. J Comput Appl Math 121:421–464
Antal E, Csendes T, Virágh J (2013) Nonlinear transformations for the simplification of unconstrained nonlinear optimization problems. Cent Eur J Oper Res 21:665–684
Bánhelyi B, Csendes T, Garay BM, Hatvani L (2008) A computer-assisted proof of Σ_3-chaos in the forced damped pendulum equation. SIAM J Appl Dyn Syst 7(3):843–867
Cambini R, Sodini C (2008) A computational comparison of some branch and bound methods for indefinite quadratic programs. Cent Eur J Oper Res 16:139–152
Casado LG, García I, Csendes T (2000) A new multisection technique in interval methods for global optimization. Computing 65:263–269
Casado LG, García I, Tóth BG, Hendrix EMT (2011) On determining the cover of a simplex by spheres centered at its vertices. J Global Optim 50:645–655
Csendes T (2001) New subinterval selection criteria for interval global optimization. J Global Optim 19: 307–327
Csendes T, Garay BM, Bánhelyi B (2006) A verified optimization technique to locate chaotic regions of a Hénon system. J Global Optim 35:145–160
C-XSC Language home page. http://www.xsc.de/
Das GK, Das S, Nandy SC, Shina BS (2006) Efficient algorithm for placing a given number of base station to cover a convex region. J Parallel Distrib Comput 66:1353–1358
Friedman E (2014) Circles covering squares web page. http://www2.stetson.edu/~efriedma/circovsqu/
Heppes A (2006) Covering a planar domain with sets of small diameter. Periodica Mathematica Hungarica 53:157–168
Heppes A, Melissen H (1997) Covering a rectangle with equal circles. Periodica Mathematica Hungarica 34:65–81
Huang WQ, Ye T (2011) Quasi-physical global optimization method for solving the equal circle packing problem. Sci China Inf Sci 54:1333–1339
Kubach T, Bortfeldt A, Gehring H (2009) Parallel greedy algorithms for packing unequal circles into a strip or a rectangle. Cent Eur J Oper Res 17:461–477
Nurmela KJ (2000) Conjecturally optimal coverings of an equilateral triangle with up to 36 equal circles. Exp Math 9:241–250
Pál L, Csendes T (2009) Intlab implementation of an interval global optimization algorithm. Optim Methods Softw 24:749–759
Palatinus E (2010) Optimal and reliable covering of planar objects with circles. In: Proceedings of the 2010 mini-conference on applied theoretical computer science
Ratschek H, Rokne J (1988) New computer methods for global optimization. Ellis Horwood, Chichester
Shimrat M (1962) Algorithm 112, position of point relative to polygon. Commun ACM 5(8):434
Vinkó T, Ratz D (2004) A multidimensional branch-and-prune method for interval global optimization. Numer Algorithms 37:391–399